

Controlling Transactions and Locks Part 5: SQL Snapshots

By Don Schlichting

Introduction

This article will explore the new Row Versioning feature in SQL, and the two transaction isolation levels that take advantage of it: Read Committed Snapshot and Snapshot Isolation.

Below are the topics that were covered in previous parts of this series.

- Locks: The SQL default method of controlling data integrity.
- Transactions: TSQL's smallest unit of work.
- ACID: An acronym used to describe a perfect and consistent transaction.
- Lock Modes: The way a lock shares or does not share records its working on.
- Blocking: When a transaction must wait for a record.
- Deadlocks: An error condition occurring when two locks are stuck in a circular loop.
- Lock Hints: TSQL commands to override SQL's default locking behavior.
- Row Versioning: The ability in SQL to keep copies of data changes.

Examples

The examples used in this article run on the SQL Adventure Works DB. Adventure Works replaces the Pubs and Northwind databases from previous SQL versions. This database can be installed from the SQL media or downloaded from Microsoft at <http://www.microsoft.com/downloads/details.aspx?familyid=E719ECF7-9F46-4312-AF89-6AD8702E4E6E&displaylang=en> .

Read Committed Snapshot

By default, all versions of SQL Server use pesismistic Read Committed locking. The first main term in this statement, "pessimistic," refers to the database expecting there to be simultaneous access conflicts, therefore, SQL is expecting conflict and will lock any row prior to access, read access or write. The second term, "Read Committed," means only values that are fully committed will be returned. If we request a row that is being modified by some other transaction, our transaction will wait until the other is finished and fully committed. This blocking action ensures only valid data is returned. However, what if we didn't want to slow down our statement by waiting on this blocking? We could use the NOLOCK hint, but then we may not be receiving valid committed data back. This situation can be addressed with a new type of Read Committed that relies on Row Versioning, called Read Committed Snapshot available in SQL. When enabled, our statement would read the last committed value out of the version history table rather than waiting on a block. In addition, our read statement will not block any other statement that is trying to write. For a description of Row Versioning, see the [forth article](#) in this series. The difference between the default locking read committed behavior and the new Read Committed Snapshot can be easily demonstrated. Open two query windows in SQL Management Studio, both using Adventure Works. This example will use the Sales.SalesTerritory table; it is a small tables containing rolled up year to date territory amounts.

Territ...	Name	Group	SalesLastYear	SalesYTD
1	Northwest	North America	3298694.4938	4358526.0429
2	Northeast	North America	3607148.9371	4825355.2670
3	Central	North America	3205014.0767	3985928.6549
4	Southwest	North America	5366575.7098	6643143.7089
5	Southeast	North America	3925071.4318	4201294.9543
6	Canada	North America	5693988.8600	6169098.6679

Enter the following uncommitted transaction as the first query.

```
USE AdventureWorks;
GO
BEGIN TRAN
UPDATE Sales.SalesTerritory
SET SalesYTD = 5000000
WHERE [Name] = 'Central';
```

In a second window, enter the following select:

```
SELECT SUM(SalesYTD) AS TotalSales
FROM Sales.SalesTerritory
```

As expected, the select statement will not respond, it is being blocked by the uncommitted update. This is the default level of SQL Read Committed using pessimistic locking. Now issue a ROLLBACK on the first statement so the second will complete. This next time, we will enable the new Read Committed Snapshot using Row Versioning and the select will execute unblocked. Execute the following statement to enable Row Versioning with the Read Committed level:

```
ALTER DATABASE AdventureWorks
SET READ_COMMITTED_SNAPSHOT ON;
GO
```

This may take several minutes. Now rerun the two queries above in the same order, the sales total 44294026.1334 should be returned. ROLLBACK the first statement. The select was not blocked because it did not wait for the first statement to commit or rollback, instead the version history table was read for the last committed value.

Updates

Even with Read Committed Snapshot enabled, Updates are handled pessimistically, using exclusive locks like SQL. Row Versioning is not used.

Multiple Statements

Using Read Committed Snapshot, each statement in a transaction checks for committed data at the start of that statement, rather than just once at the beginning of the entire transaction. For example, if our above select example was run several times in a row inside

a transaction, and in between statement executions some other transaction changed the total sales value, this new value would be reflected in our next return.